M3：项目上下文与技能

资源发现、渐进加载与不可变快照

M3：项目上下文与技能
- 背景、目标与非目标
- 行为规格
  - M3-S01：工作边界与信任
  - M3-S02：上下文文件发现
  - M3-S03：技能发现与优先级
  - M3-S04：SKILL.md 格式
  - M3-S05：自动索引与 prompt 组装
  - M3-S06：显式技能命令
  - M3-S07：资源快照与刷新
  - M3-S08：错误与诊断
- 数据结构与公共 API
- 详细设计与模式
  - 资源库与优先级
  - 不可变快照
  - 渐进加载
- 验收示例
- 逐步资源示例
  - 路径与覆盖
  - 快照与历史
  - 显式调用序列化
  - 资源诊断的层次
- 备选方案与风险

M3：项目上下文与技能

属性	内容
状态	设计提案 1.0
前置	M2的 context entry
交付	`@local/resources`、`/reload`、`/skill:<name>`
验收	M3-A01 至 M3-A10

背景、目标与非目标

模型需要项目约束，但把所有文档一次塞进上下文会增加成本并挤占任务历史。项目说明通常应始终可见；专门技能适合先提供摘要，再按需读取正文。会话历史、项目说明和技能是三种不同来源，不能统称为一个可随意覆盖的“记忆”。

本模块负责资源发现、校验和 prompt 组装，输出不可变快照；不运行模型、不写日志、不执行技能里的命令。非目标是 embedding、自动长期记忆、完整 Git ignore 语义和可执行插件；插件见 M5。

行为规格

M3-S01：工作边界与信任

CLI 提供已 realpath 的 workspace、cwd、agentHome 和 projectTrusted。cwd 必须在 workspace 内。workspace 是工具访问边界；资源查找可以向上寻找最近 Git 根作为 stopAt，但不得越过显式 workspace。如果没有 Git，stopAt=workspace。Git 元数据在 M4 通过注入函数接入；M3 可以直接传 stopAt 测试，无需提前实现 Git。

未信任时不得 readdir/stat/read 项目 `.terminal-agent/skills`、`.agents/skills`、extensions 和 mcp.json，也不自动读取项目上下文文件；用户级资源仍加载。信任必须由 `--trust-project` 明确提供。显式 `--context-file <path>` 是单文件读取授权，不连带信任其邻居目录。

资源扫描不跟随符号链接。单个文件上限 256 KiB，全部自动上下文文件累计上限 1 MiB，最多 1000 个技能、扫描深度 16。超限跳过对应项并产生诊断，不能加载部分正文冒充完整资源。

M3-S02：上下文文件发现

每个目录最多取一个普通文件，优先级固定：`AGENTS.override.md`、`AGENTS.md`、`CLAUDE.md`。大小写敏感，不再枚举任意变体。同目录取第一个存在且合法的；若第一个存在但不可读/超限，记诊断，不静默退回另一个文件。

顺序为：agentHome 根说明 → stopAt → 沿路径逐级到 cwd → 显式 context-file 参数顺序。项目祖先必须位于 workspace，stopAt 包含在内。去重使用 realpath，相同文件只保留首次。文件去 UTF-8 BOM，CRLF 规范为 LF；其他文本保留。

所有文件作为带 path 的独立段落组合。后面的文件距离任务更近；prompt 说明冲突时优先更近项目说明，显式用户任务仍是任务输入。程序权限永远由配置控制，文档不能给自己增加 bash 或外部目录权限。

M3-S03：技能发现与优先级

扫描源优先级如下，先发现者获胜：

次序	路径	条件
1	cwd 到 stopAt，逐层 `.terminal-agent/skills`	projectTrusted
2	cwd 到 stopAt，逐层 `.agents/skills`	projectTrusted
3	`<agentHome>/skills`	总是
4	`~/.agents/skills`	总是
5	`--skill-path` 参数给出的文件或目录	显式选择，按参数顺序

同一源内按相对路径码点排序，深度优先。目录内有 SKILL.md 时作为一个技能根，不进入该目录子树；否则继续下钻。显式文件必须名为 SKILL.md；不把普通 Markdown 隐式当技能。跳过 `.git`、`node_modules`、隐藏目录和符号链接，扫描源目录本身的前导点不触发跳过。此基线不解析 `.gitignore`，避免半套规则造成误解。

同名技能保留先发现的，产生 collision 诊断，列 winnerPath/loserPath。相同 realpath 的重复源直接去重，不产生 collision。错误的一个技能不能阻断其他技能扫描。

M3-S04：SKILL.md 格式

文档必须以独占一行的 `---` 开头；在后续独占一行的 `---` 结束 YAML frontmatter。没有分隔符、YAML 语法错、重复键或非对象 frontmatter 都是无效技能。使用只解析数据的 YAML 库，不接受自定义可执行 tag。未知键忽略并给 warning，不能执行其中任何值。

字段	类型和约束	缺省
name	1..64 ASCII 字符；小写字母/数字/单连字符；不首尾连字符	技能目录 basename
description	非空 string，trim 后不超过 1024 字符	必填
disable-model-invocation	boolean	false

name 还不能包含 `--`。description 用于模型发现；body 是结束分隔符后的原文，去除最前一个 LF，必须含非空白。name/description 的解析错误令整个技能无效；boolean 字段的字符串 `"false"` 不隐式转型。

```
---
name: review-code
description: Review a source change and list concrete issues.
disable-model-invocation: false
---
Read the changed files. Check behavior and failure cases.
Return findings with file paths and reasons.
```

扫描只缓存元数据和正文 hash，不把 body 放进自动 prompt；显式调用时重新读取并校验一次，避免扫描后文件被移走或变为符号链接。

M3-S05：自动索引与 prompt 组装

`buildPrompt(snapshot,{hasReadTool,toolNames})` 是纯函数。固定段顺序：preamble → tool names → project context → skill index → cwd。每段都有固定标题，空可选段完全省略。preamble 为 `You are a coding assistant. Use tools to inspect facts before changing files.`；tool names 排序，不能由 description 猜测不存在的能力。

只有存在名为 read 的有效文件工具时才加入自动 skill index；M3 单独验收可以注入 `hasReadTool:true`，M4 正式接入。禁止把 disable-model-invocation=true 的技能放入自动索引，但保留在显式命令表。

索引包括 name、description、location，不含正文。使用 JSON 数组序列化，以 JSON.stringify 转义路径和文本，避免用正则解析嵌套 XML。可在前面解释“匹配任务时先用 read 读取 location”。项目上下文也以 `{path,content}` 数组序列化；这些是模型数据，不是可执行配置。

M3-S06：显式技能命令

输入语法为 `/skill:<name>`，可追加一个 ASCII 空格和任意任务文本。只识别从第一列开始的完整命令；普通句子中的 `/skill:` 不触发。name 使用同一验证规则；剩余任务保留原字符串，不支持 shell 引号或参数展开。

CLI 在 idle 时查当前快照，未知 name 返回 NOT_FOUND，不调用模型、不写用户消息。读取正文并复核文件范围、类型和大小；失败返回 NOT_FOUND/INVALID_INPUT，快照标过期。成功将 user.text 构造成一个 JSON 对象字符串：

`{kind:'skill_invocation',name,location,instructions,task}`，其中 instructions 为去 frontmatter 后正文，task 缺省为空字符串。

这是普通用户消息，进入 M1 prompt 和 M2 日志。技能内容不提高角色等级，也不把 allowed-tools 字段转成授权。日志保存调用时正文，恢复不重新读取当前文件，因此历史含义稳定。

M3-S07：资源快照与刷新

`scanResources(options): ResourceSnapshot` 返回 contextFiles、skills、diagnostics、fingerprint。fingerprint=规范化 prompt 输入（包含路径、规范化内容、技能元数据及 bodyHash）的 SHA-256，不含 clock、扫描耗时或诊断排序噪声。即使只改技能正文，fingerprint 也变化。

每次接受新 prompt 前，CLI 在 idle 扫描并比较 fingerprint；有变化时先构造完整新 prompt，再调用 Session.setContext。写入成功才替换进程里的 active snapshot。失败保持旧快照并拒绝这次任务，不半更新。

`/reload` 是同样流程，执行成功输出加载计数和诊断，不调用模型、不产生 user 消息。运行中返回 BUSY。连续两次无变化不追加 context entry。M5 的工具重载额外改变 toolsFingerprint，一并原子提交，不能只改 prompt。

M3-S08：错误与诊断

路径不存在的默认可选目录安静跳过；用户显式给出的路径不存在必须 warning。不可读、超限、YAML 错、重名、越界均有结构化诊断，含 code/path/message，collision 带 winner/loser。扫描结果按 source 优先级和 path 排序，便于复现。

不得把原文或密钥全量写到 stderr。资源诊断不是 run error；只要剩余资源可用，任务可继续。显式技能调用失败则不提交任务，不能悄悄退回普通 prompt。

数据结构与公共 API

```
type ContextFile = {
  path: string;
  content: string;
  source: "user" | "project" | "explicit";
  depth: number;
};
type Skill = {
  name: string;
  description: string;
  path: string;
  baseDir: string;
  scope: "user" | "project" | "explicit";
  disableModelInvocation: boolean;
  bodyHash: string;
};
type ResourceDiagnostic = {
  code: "UNREADABLE" | "INVALID_SKILL" | "LIMIT" | "COLLISION" | "SKIPPED";
  path: string;
  message: string;
  winnerPath?: string;
  loserPath?: string;
};
type ResourceSnapshot = {
  contextFiles: readonly ContextFile[];
  skills: readonly Skill[];
  diagnostics: readonly ResourceDiagnostic[];
  fingerprint: string;
};
type ScanOptions = {
  workspace: string;
  cwd: string;
  stopAt: string;
  agentHome: string;
  projectTrusted: boolean;
  skillPaths: readonly string[];
  contextPaths: readonly string[];
};
```

API	输入输出	副作用
scanResources	ScanOptions → Promise	只读文件
parseSkill	文本 + path → Skill 元数据/body 或诊断	无
buildPrompt	snapshot + 工具描述 → string	无
resolveInvocation	snapshot + 输入 → 已展开文本或非命令	显式命令读取一次文件

resources 不 import Core 或 Session 类型。CLI 的组合根（Composition Root）把工具表投影为 string[]，把生成 prompt 交给 Session。

详细设计与模式

资源库与优先级

将发现与解析分开：scanner 只产出候选文件及来源序号；parser 只处理单文件；resolver 用 `Map<name,Skill>` 执行先发现优先；formatter 只处理合法值。这是 Repository 思路的文件版本，不需要建数据库或通用仓储框架。

上下文沿目录链做有序列表，技能以 Map 去重后输出排序数组。扫描 f 个候选、b 字节总内容，排序成本 O(f log f)，解析 O(b)，存储 O(b + f)。元数据读取不能悄悄缓存为永久结果，除非有明确失效逻辑；教学基线每次 idle 输入前重新扫描。

不可变快照

快照一次扫描生成，使用后不修改。刷新生成候选快照、构建 prompt、持久化 context、交换指针，形成小型两阶段发布。不能先更新 activeSkills 再写 prompt，否则写失败后技能命令表与会话上下文不一致。

恢复使用日志中的旧 prompt；新输入前才刷新本地文件。这样“查看历史”不改变历史，“继续任务”能使用当前资源。快照不是文件系统原子快照：扫描期间文件可能变化，显式调用的二次校验负责发现差异，不能保证多个文件来自同一磁盘瞬间。

渐进加载

索引是目录，正文通过 read 或显式技能命令进入历史。这降低默认上下文大小，并保留模型实际读到的内容。read 工具在 M4 仅能读工作区；用户级 skill 通常在工作区外，因此 CLI 必须向 read 提供精确的只读资源文件 allowlist，仅包含当前已扫描技能的真实路径，不开放整个 home。read 在该清单中可读，write/edit/搜索不继承授权。

允许列表属于程序能力配置，不从技能文字解析。reload 后仅在下一 run 更新工具快照，避免活动请求读到突然变化的权限范围。

验收示例

假设 user skill 名为 review，工作区近层也有 review，远层有 lint：可信项目取近层 review 并报一次 collision；索引含 review/lint 的描述和路径，不能含正文。关闭 projectTrusted 后项目目录扫描次数必须为零，只剩 user review。

编号	场景	断言
M3-A01	同目录 override/AGENTS/CLAUDE 同时存在	只取 override；文件顺序 user→root→cwd

编号	场景	断言
M3-A02	未信任项目	项目资源路径无 stat/readdir/read；用户资源仍在
M3-A03	近/远层和用户目录同名技能	近层先胜；collision 记录准确
M3-A04	缺 description、重复 YAML 键、非法 name	无效项跳过；其他项不丢
M3-A05	SKILL.md 根含下级技能和 symlink 环	不递归技能根；不跟随 link；扫描有界
M3-A06	禁用自动调用、有/无 read 工具	显式表存在；自动索引按条件隐藏
M3-A07	`/skill:review` 改进函数、未知技能、正文含换行/引号	正文 JSON 可往返；未知不调用模型
M3-A08	连续 reload；修改 body 再 reload	不变不落盘；bodyHash 变化触发新 context
M3-A09	刷新持久化失败或 busy	旧快照保持；无半更新
M3-A10	用户技能路径在 workspace 外	read 仅获指定文件权限；写工具仍 PATH_ESCAPE

测试使用临时目录和内容快照；涉及项目未信任的测试要记录文件系统端口调用，而不只检查最终数组。

逐步资源示例

路径与覆盖

workspace=`/w`，cwd=`/w/src`，agentHome=`/h`，且可信。`/h/AGENTS.md`、`/w/AGENTS.md`、`/w/src/AGENTS.override.md` 同时存在，prompt按这一顺序保留三个文件，不把它们合并成同一字符串覆盖。`/w/src/AGENTS.md` 被同目录override遮蔽，产生的信息不进入prompt。

若技能review在`/w/src/.terminal-agent/skills/review/SKILL.md`、`/w/.terminal-agent/skills/review/SKILL.md`和`/h/skills/review/SKILL.md`各有一份，近层胜出，两个输家各一条collision。disabled字段来自赢家，不从输家合并。

快照与历史

第一次prompt时description为“检查函数”，bodyHash=H1。运行期间文件被改成“检查数据库”；当前run仍使用旧工具/资源快照，已接受的显式技能也保持当时正文。下一次idle输入前重新扫描生成H2，先写context再开始新用户消息。恢复旧会话仅查看历史时不扫描重写，避免历史显示随本地文件变化。

`/reload`没有变化时不产生entry；有变化时只产生一条完整context。context写失败不得把新的技能命令表发布给CLI，否则接下来`/skill:name`可能引用模型尚未获知的资源版本。

显式调用序列化

技能路径或正文含引号、反斜线、`</skill>`等文本时仍通过JSON.stringify保存。解析或显示时把它当文本值，不运行shell、不按HTML标签截断。调用形成的user消息可以直接持久化；不额外保存一个与它内容重复的“技能消息”entry。

资源诊断的层次

问题	自动发现时	显式命令时
默认技能目录不存在	安静略过	不适用
用户给定路径不存在	warning，其他资源可用	NOT_FOUND，不提交任务
YAML无效/字段超长	跳过该技能并诊断	INVALID_INPUT，不提交任务
同名技能	保留赢家，报collision	使用当前赢家，不猜用户指哪一个
文件扫描后变link	下次扫描跳过	复核失败，不能读取新目标
project未信任	完全不遍历	仅在明确skill-path授权文件时可加载

显式路径授权不意味着默认信任整个项目。对于重复realpath、名称冲突和路径排序分别保留测试，不能用一个大快照掩盖三种不同规则。

备选方案与风险

一次性加载所有正文简单，但长会话成本不可控，且不知道模型实际使用哪个技能。技能作为 executable plugin 会带来不同生命周期和权限，另放 M5。用 XML 正则包正文容易被结束标签和路径引号破坏，选定 JSON 序列化并由程序解析。

项目说明可能含错误或注入文本。分层 prompt 只表达信息来源，不是安全隔离；权限仍由工具和信任配置强制执行。任何文件内容都不能改变 read allowlist 或让程序自动执行扩展。

实施顺序：单文件解析 → 目录候选和排序 → 冲突解析 → prompt 纯函数 → 快照发布 → 显式命令 → 恢复后刷新。M3 可用假工具声明验收，M4 才把正文按需读取连到真实文件工具。